GIT

LOGICIEL DE VERSIONNING



No notes on this slide.

GIT

Rappel

Git est un outil de *versioning*, il permet de résoudre 3 problèmes de gestion de code :

1. Garder un historique des modifications et de l'évolution du code
2. Sauvegarder le code en lieu sûr
3. Collaborer facilement

Git est un logiciel open source, disponible sur tous les systèmes

Git est un incontournable du développement logiciel. C'est un programme open source utilisé partout dans le monde. Il existe plusieurs alternatives à git (Perforce, Mercurial, etc) mais git est le plus utilisé, particulièrement dans le monde open-source. Git gère tous les types de données textuelles, pas seulement du code. Des livres, des documentations, des configurations, des notes, etc peuvent très bien être des projets gérés par git. Par exemple, ces slides utilisent git et sont stockées sur GitHub. En revanche git n'est pas fait pour gérer des fichiers compressés ou binaires, car ces derniers changent beaucoup d'une version à une autre.

GIT

Notions de base

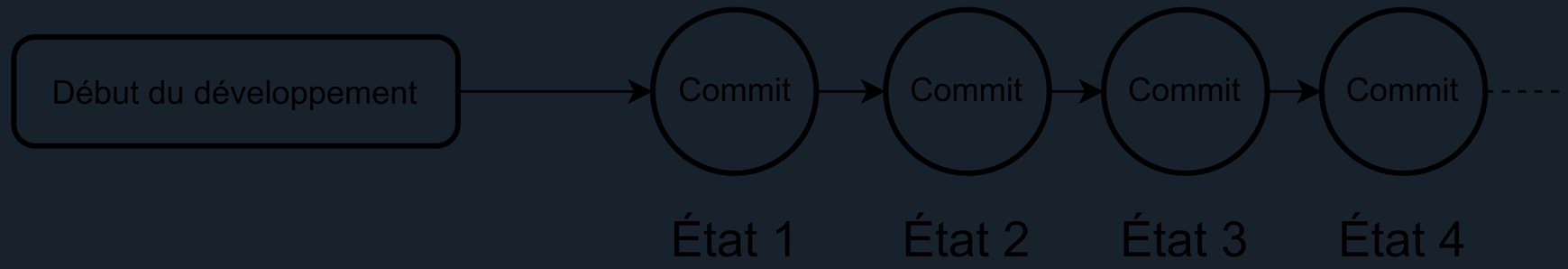
La notion fondamentale dans git est le *commit*.

Il s'agit d'un instantané de votre projet, un "point de sauvegarde" qui enregistre son état.

Git permet de revenir, de comparer, de mélanger différents commits.

Ce qui se passe entre les commits n'est pas sauvegardé.

Git enregistre l'ordre des commits, qui sont liés les uns aux autres.



Par exemple la liste des derniers commits de QGIS : https://github.com/qgis/QGIS/commits/master/

Quand un commit est enregistré, git attribue un identifiant unique au commit et enregistre sa différence par rapport au commit précédent. Tout est placé de manière transparente dans le dossier .git de votre projet, qui est un dossier caché.
On ne modifie jamais le contenu du dossier .git manuellement. Tout se fait via des commandes dans le terminal.

GIT

Commit et tracking

Dans un projet, pour indiquer à git qu'un fichier doit être tracké (pris en compte), la commande est :

```
git add ./chemin/vers/le/fichier.txt
```

Pour faire un commit, la commande est :

```
git commit -am "Message"
```

Git créé alors un nouveau commit

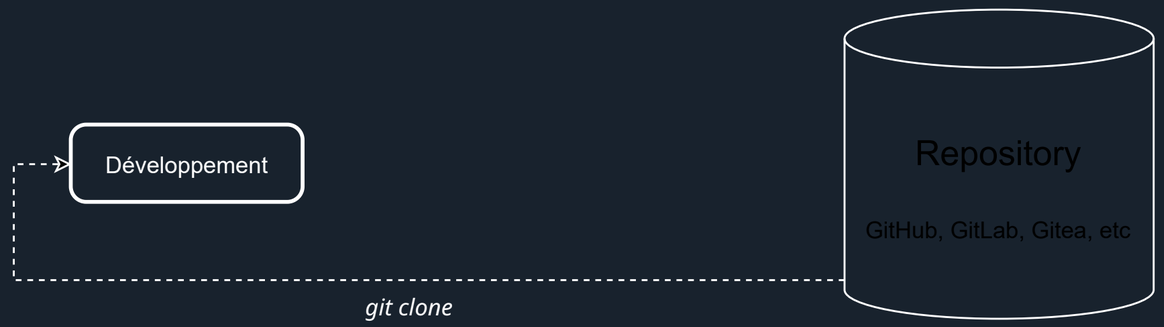
- L'option -a indique de prendre tous les fichiers trackés modifiés
- L'option -m indique le message du commit. Il s'agit d'un texte décrivant ce qui a été modifié dans ce commit.

Si l'option -m n'est pas indiquée, git ouvrira un éditeur de texte pour entrer le message de commit

Par défaut, git n'enregistre pas l'ensemble des fichiers. Il faut lui indiquer quels fichiers sont importants avec la commande git add La commande git status permet de voir l'état actuel d'un projet. Elle indiquera les fichiers modifiés depuis le dernier commit, et s'ils sont trackés ou non. Les messages de commit doivent être courts et le plus descriptif possible. Mauvaise exemple : "J'ai modifié plusieurs fichiers de la configuration globale pour la prochaine mise à jour, et corrigé plusieurs bug divers mais pas très importants" (C'est long et très peu précis) Bon exemple : "Ajoute la couche bâtiments de OpenStreetMap" (Court et précis) Il est également possible de ne commit que certains fichiers modifiés, quand on ne précise pas l'option -a, mais ceci sort du cadre de ce cours.

GIT

Notions de base



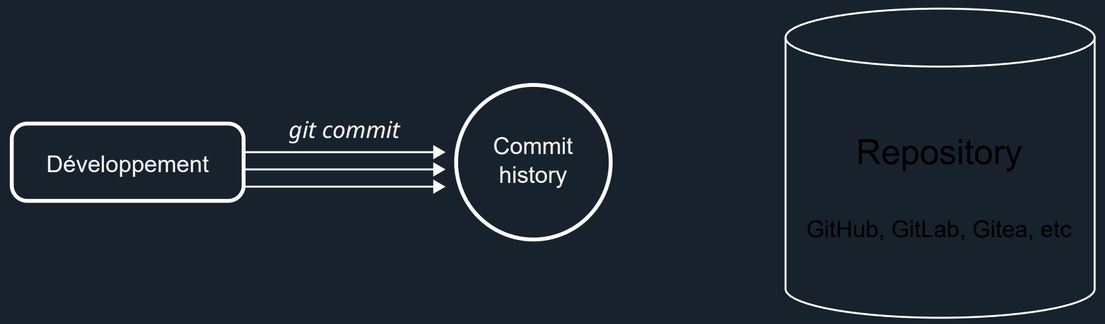
Pour contribuer à un projet existant, on récupère le code sur un *dépôt* distant (*repository* en anglais). On dit que l'on *clone* un dépôt distant vers un dépôt local.

Cette étape ne se fait qu'une seule fois.

No notes on this slide.

GIT

Notions de base

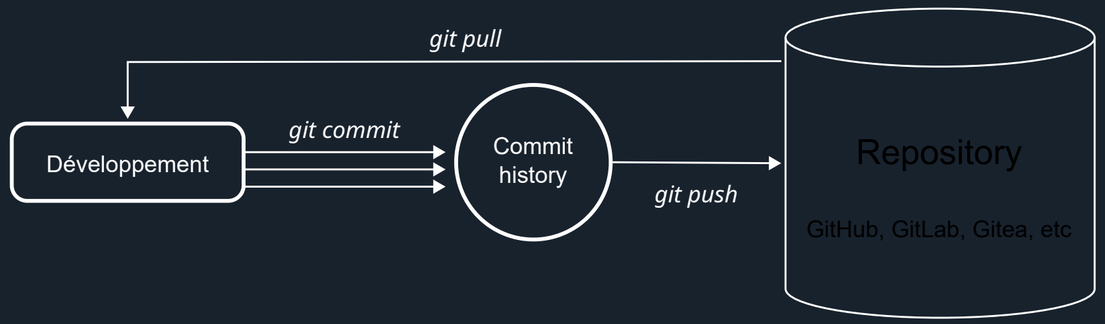


Depuis ce dépôt local, on fera des modifications sur le code, avec les commits associés.

No notes on this slide.

GIT

Notions de base



Pour synchroniser nos changements avec le dépôt distant, on fera un *push*. Git mettra à jour le dépôt distant avec nos nouvelles modifications.

Si de nouvelles modifications ont été ajoutées par d'autres personnes dans le dépôt distant, on pourra les récupérer à l'aide de la commande *pull*.

Attention : Vous ne pouvez `push` que si vous avez la dernière version du code, sinon il vous faudra `pull` et mettre votre code à jour

No notes on this slide.

GIT

Workflow

Les commandes principales

- `git clone` : récupère le code depuis un dépôt distant
- `git add <file>` : ajoute un fichier au prochain commit
- `git commit -m "<message>"` : créé un commit avec son message associé
- `git push` : envoie vos nouveaux commits dans le dépôt disant
- `git pull` : récupère les nouveaux commits depuis le dépôt disant

La commande `git commit -am "<message>"` permet de créer un commit avec tous les fichiers modifiés, elle est équivalente à `git add` de tous vos fichiers, puis faire un commit.

La commande `git status` affiche l'état actuel de votre projet (fichiers modifiés, ajoutés au prochain commit, etc)

Il y a beaucoup d'autres commandes, nous ne voyons ici que les commandes les plus basiques.
La commande `git add` permet d'ajouter tous les fichiers qui correspondent à un critère. Par exemple `git add *.js` ajoutera tous les fichiers qui finissent par ".js". La commande `git add .` ajoutera tous les fichiers du dossier courant et de ses sous-dossiers au prochain commit.

GIT

Les conflits

Supposons :

1. Alice et Bob clonent leur projet, chacun de son côté
2. Alice écrit `div{ color:blue; }` , puis fait un commit
3. Bob écrit `div{ color:green; }` , puis fait un commit
4. Bob push son code avec *git push*
5. Alice push son code avec *git push*

Que se passe-t-il ?

No notes on this slide.

GIT

Les conflits

Supposons :

1. Alice et Bob clonent leur projet, chacun de son côté
2. Alice écrit `div{ color:blue; }` , puis fait un commit
3. Bob écrit `div{ color:green; }` , puis fait un commit
4. Bob push son code avec *git push*
5. Alice push son code avec *git push*

Que se passe-t-il ?

```
 ! [rejected] main -> main (fetch first)
error: failed to push some refs to 'https://github.com/...'
```

Alice ne peut pas push, car elle ne possède pas la dernière version du code. C'est le commit de Bob qui est le plus récent. Que doit faire Alice ?

Au moment d'executer le point 5, git effectue un push. Mais le code présent sur le repository est plus récent que le code que possède Alice (car le code du repository contient le commit de Bob).
Git ne va jamais écraser un changement, ce serait beaucoup trop dangereux, git refuse alors de faire le push. En pratique git indique toujours dans le message d'erreur ce qu'il est conseillé de faire. Voir slide suivante...

GIT

Les conflits

La solution : Alice doit d'abord faire un `git pull` pour récupérer la dernière version du code.

A ce moment, git va mélanger (*merge*) les deux versions du code, celle d'Alice et celle de Bob.

Deux cas peuvent se produire :

No notes on this slide.

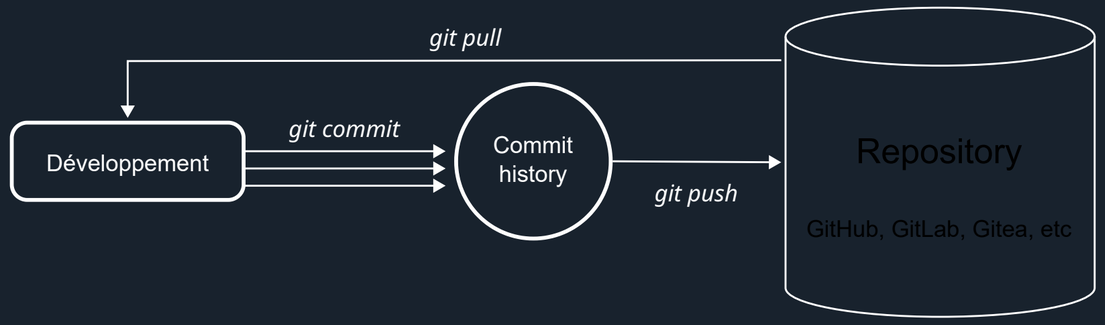
GIT

Les conflits

Cas 1 :

Les modifications ne sont pas contradictoires, git parvient à faire automatiquement le merge

- Alice possède alors une version du code mélangée
- Il lui suffit de faire un `git commit` et un `git push` pour push sa nouvelle version.



git travaille par différence entre les fichiers. Si deux fichiers différents ont été modifiés, alors les modifications de chacun seront fusionnées. Si le même fichier a été modifié, et qu'il s'agit de lignes différentes, là aussi il y a fusion. En revanche, si la même ligne d'un même fichier a été modifiée, alors les modifications sont "contradictoires", et git ne sait pas quelle version choisir.

GIT

Les conflits

Cas 2 : Il y a conflit !

Lors du *git pull* Alice recevra le message :

```
Auto-merging style.css
CONFLICT (content): Merge conflict in style.css
Automatic merge failed; fix conflicts and then commit the result.
```

Dans le fichier concerné, ici style.css, les conflits sont indiqués sous la forme :

```
<<<<<<< HEAD
div{ color:blue; }
=======
div{ color: green; }
>>>>>>> b6eeeaef7d4c17e8b7ad2b90968e2d17720ba319
```

Alice devra alors résoudre les conflits manuellement, puis `git commit` et `git push`

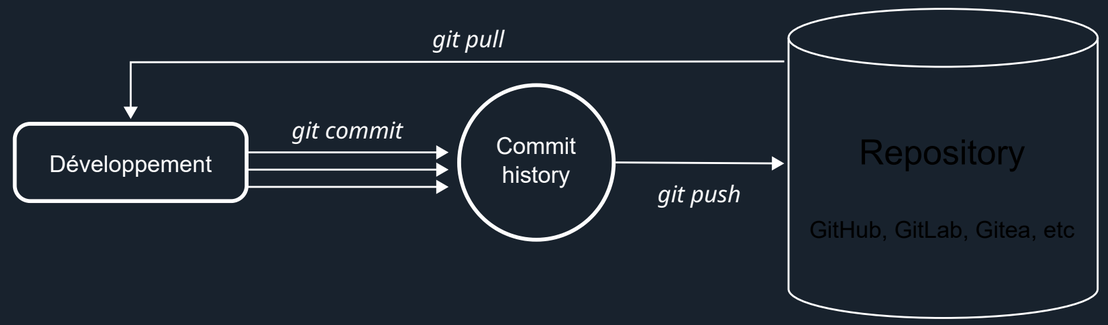
Git indique les conflit en commençant par la version distante du code ("incomming change") et ensuite la version locale du code ("current change"). HEAD indique quelle est la version distante. Dans ce cas il s'agit du tout dernier commit effectué dans le repository, appelé "HEAD". "b6eeeaef7d4c17e8b7ad2b90968e2d17720ba319" indique le hash du commit. Chaque commit dans git possède un identifiant unique appelé son hash. Ici il y a donc un conflit entre le HEAD et le commit b6eeeaef...

GITHUB

Les dépôts

GitHub est une plateforme en ligne qui permet d'héberger du code. Elle fournit un stockage pour des dépôts git.

Il existe d'autres alternatives, par exemple GitLab (hébergement de code en ligne), ou Gitea (hébergement à faire sois-même, *self-hosted*).



Les dépôts en ligne fournissent également plein de services complémentaires (gestion de bugs, planification, droits d'accès, etc).

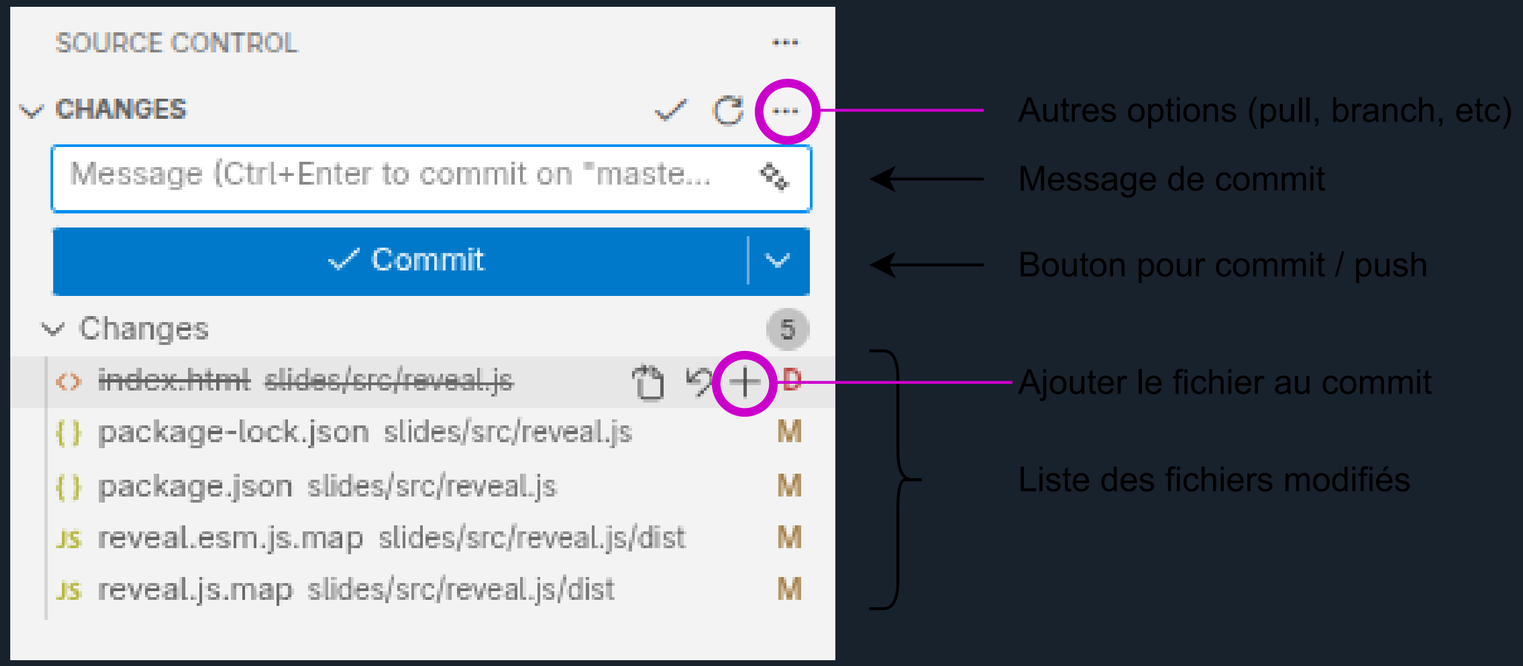
Aujourd'hui les fournisseurs de stockage proposent de nombreuses options en plus du simple stockage. GitHub permet d'executer des actions sur le code (déploiements, compilations, etc), de faire des analyses de sécurité, de tenir une liste des tâches, et bien d'autres fonctions. GitHub affichera, sous la liste des fichiers et dossiers, le contenu du fichier "README.md" du dossier courant. C'est particulièrement pratique pour créer une page d'accueil de votre projet.

GIT

Avec Visual Studio Code

Visual Studio Code détecte automatiquement les changements dans un dépôt local.

Ils sont affichés dans le menu 



Si aucun fichier n'est ajouté au commit, VS Code proposera d'ajouter tous les fichiers modifiés.

L'interface de VS Code pour gérer git est très pratique, n'hésitez pas à l'utiliser. Toutefois, il faut connaître les commandes git pour comprendre ce que l'ont fait et régler des situations que VS Code ne permet parfois pas de résoudre.
D'autres éditeurs de code fournissent des fonctions similaires.

GIT

Le fichier .gitignore

Pour exclure totalement un fichier, un ensemble de fichiers, ou un répertoire d'un projet git, cela se fait grâce à un fichier spécial : le *.gitignore*

```
# contenu de .gitignore
.env                # Ignore le fichier ".env"
*.pyc               # Ignore tous les fichiers d'extension .pyc
dossier/a/exclure/  # Ignore le dossier dossier/a/exclure/ et tout son contenu
```

Cela est particulièrement utile pour exclure les fichiers d'environnement et les fichiers temporaires générés par des compilateurs et autres outils de travail.

Les fichiers ignorés seront ignorés par la commande `git add` et ne seront pas affichés dans `git status`. Git se comportera comme s'ils n'existaient pas.

Attention : Si vous avez déjà ajouté un fichier / dossier au projet, l'indiquer dans le .gitignore n'aura aucun effet

No notes on this slide.

GIT

Resources

Site officiel : https://git-scm.com/

Livre officiel, gratuit : https://git-scm.com/book/en/v2

Tutoriel simple, pas à pas, partant de zéro : https://www.w3schools.com/git/default.asp

Et aussi un très grand nombre d'autres ressources en ligne, textes, vidéos, slides, etc

No notes on this slide.